Project 8: Design and Deployment of a Highly Available and Scalable 3-Tier Web Application on AWS

Project Description

This project demonstrates the design and deployment of a highly available and scalable three-tier web application on Amazon Web Services (AWS). A custom Virtual Private Cloud (VPC) was created with public, private application, and private database subnets. The application consists of a React frontend, Node.js backend, and Amazon RDS MySQL database. High availability was achieved using an Application Load Balancer (ALB), Auto Scaling Group (ASG), and a Golden AMI. The application was successfully deployed and accessed through the Load Balancer DNS.

Prerequisites

- AWS Account
- Amazon VPC
- Amazon EC2
- Amazon RDS (MySQL)
- Internet Gateway
- NAT Gateway
- Route Tables
- Security Groups
- Application Load Balancer
- Target Group
- Launch Template
- Auto Scaling Group
- Amazon Machine Image (AMI)

- Git
- Node.js
- React
- Nginx
- PM2
- Basic AWS Networking Knowledge

Implementation Steps

1. Create a custom Virtual Private Cloud (VPC).
2. Create public, private application, and private database subnets across multiple Availability Zones.
3. Attach an Internet Gateway to the VPC and configure a NAT Gateway for outbound internet access from private subnets.
4. Configure Route Tables for public, application, and database subnets.
5. Create Security Groups for the Application Load Balancer, EC2 instances, and Amazon RDS.
6. Launch an Amazon RDS MySQL database instance inside the private database subnets.
7. Launch a Builder EC2 instance and install Git, Node.js, Nginx, and PM2.
8. Clone and deploy the React frontend and Node.js backend application.
9. Configure Nginx as the web server and verify backend connectivity with Amazon RDS.
10. Create a Golden AMI from the Builder EC2 instance.
11. Create a Launch Template using the Golden AMI.
12. Configure an Auto Scaling Group using the Launch Template.
13. Create an Internet-facing Application Load Balancer and configure a Target Group.
14. Register Auto Scaling instances with the Target Group and verify successful health checks.
15. Access the application using the Application Load Balancer DNS and verify successful deployment.

Result

The highly available and scalable three-tier web application was successfully deployed on Amazon Web Services (AWS). A secure infrastructure was created using a custom VPC with public and private subnets. The React frontend, Node.js backend, and Amazon RDS MySQL database communicated successfully. High availability and scalability were achieved using an Application Load Balancer, Launch Template, Golden AMI, and Auto Scaling Group. The application was successfully accessed through the Application Load Balancer DNS, confirming successful deployment.

Screenshots and implementation evidence

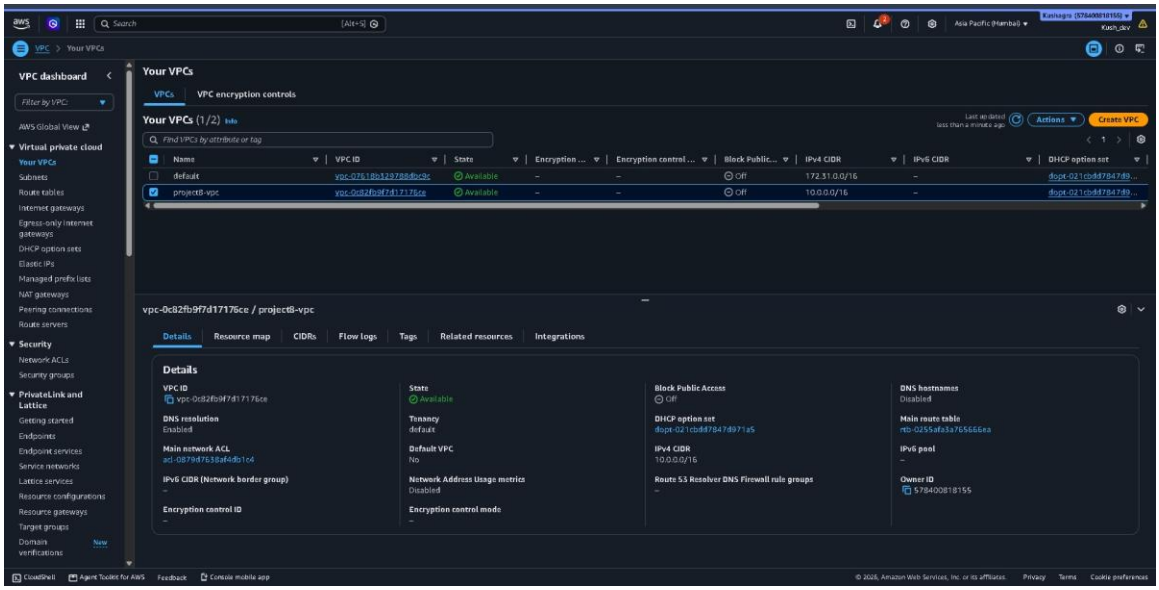

Figure 1: Virtual Private Cloud (VPC) Configuration

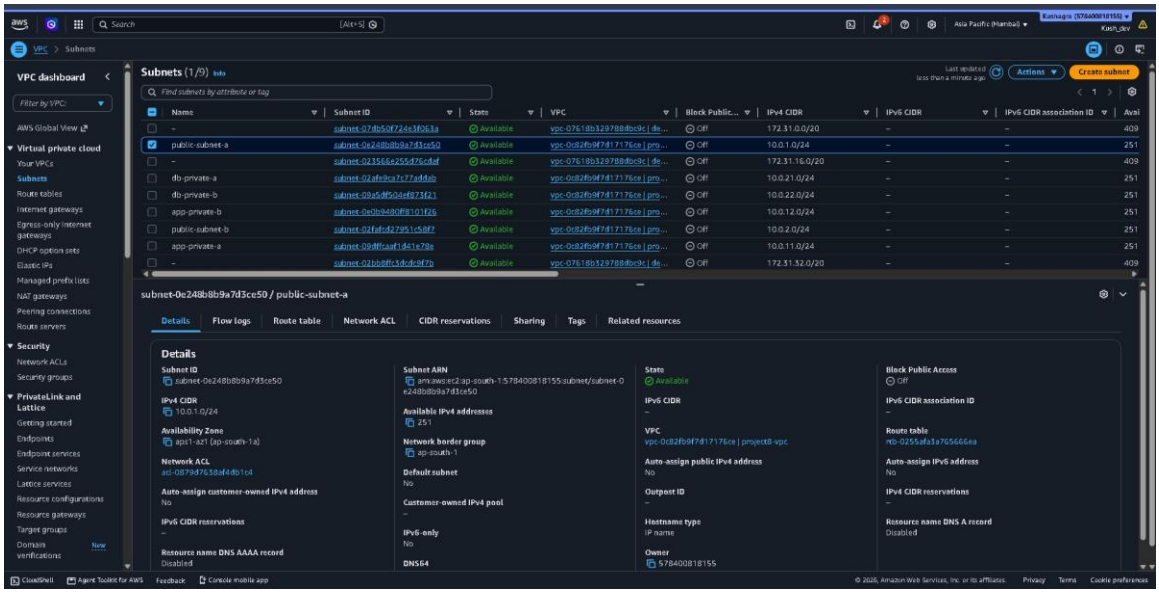

Figure 2: Public and Private Subnets Configuration

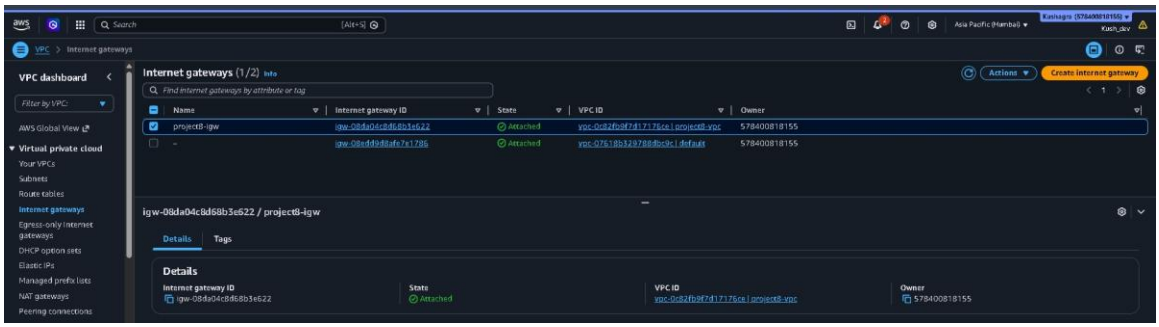

Figure 3: Internet Gateway Configuration

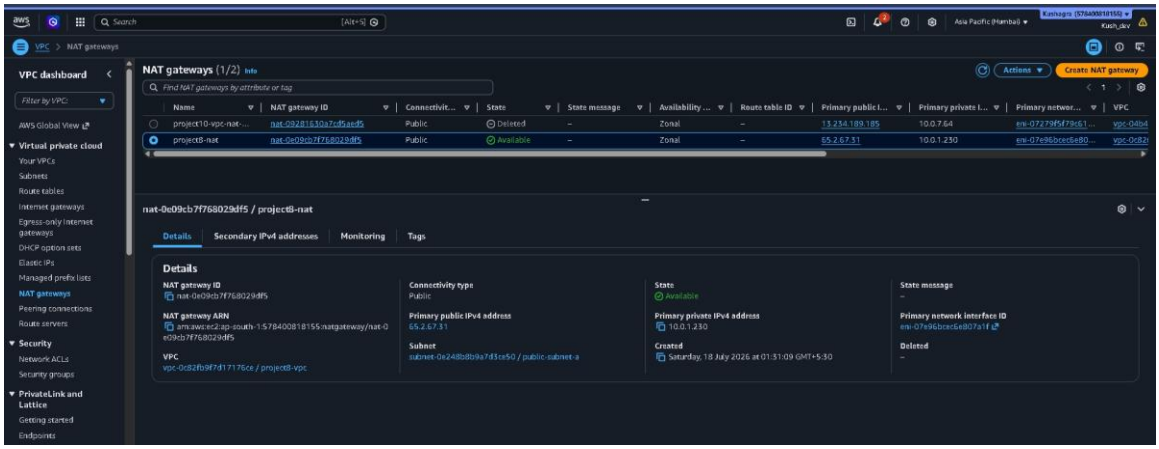

Figure 4: NAT Gateway Configuration

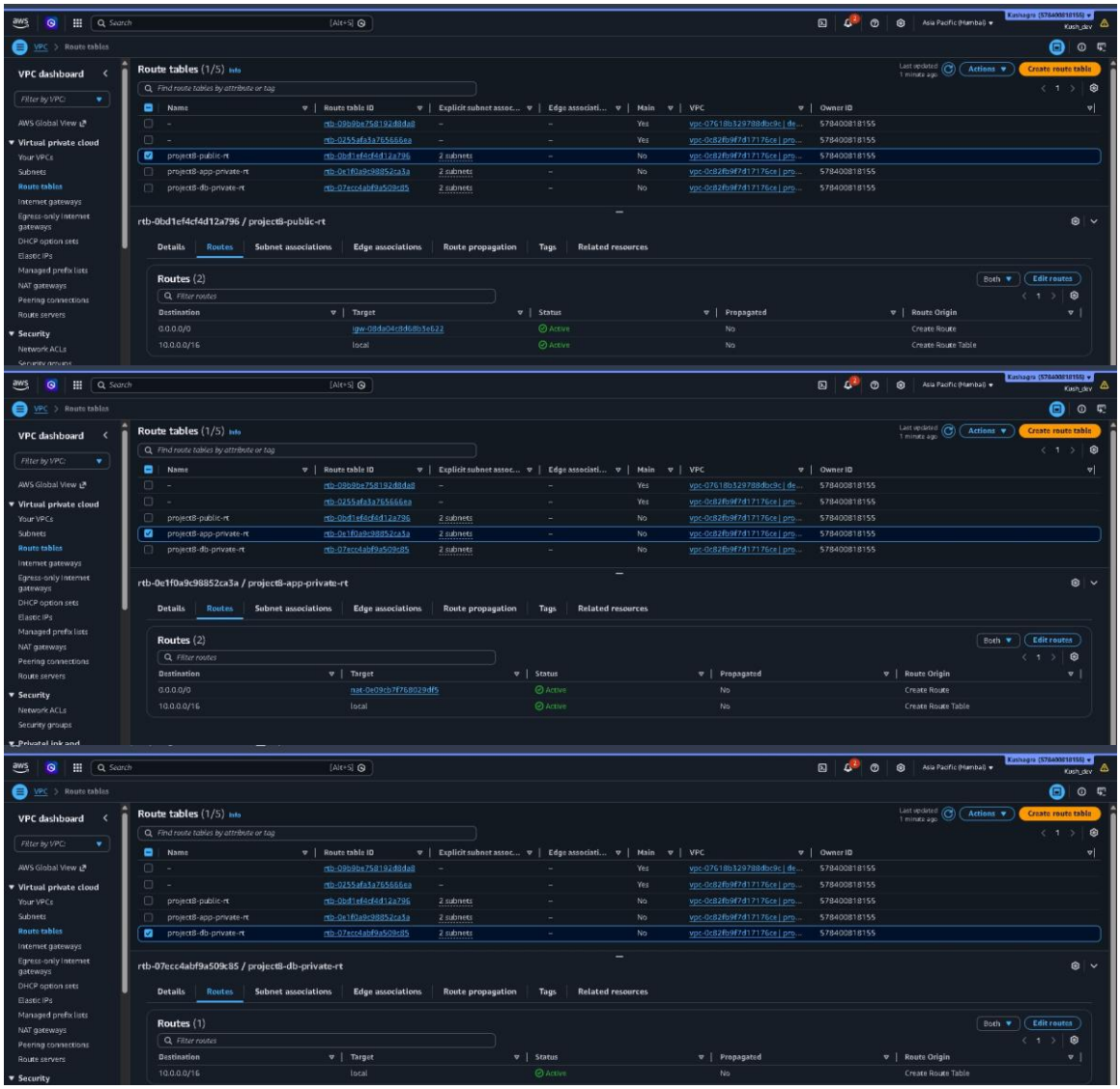

Figure 5: Route Table Configuration

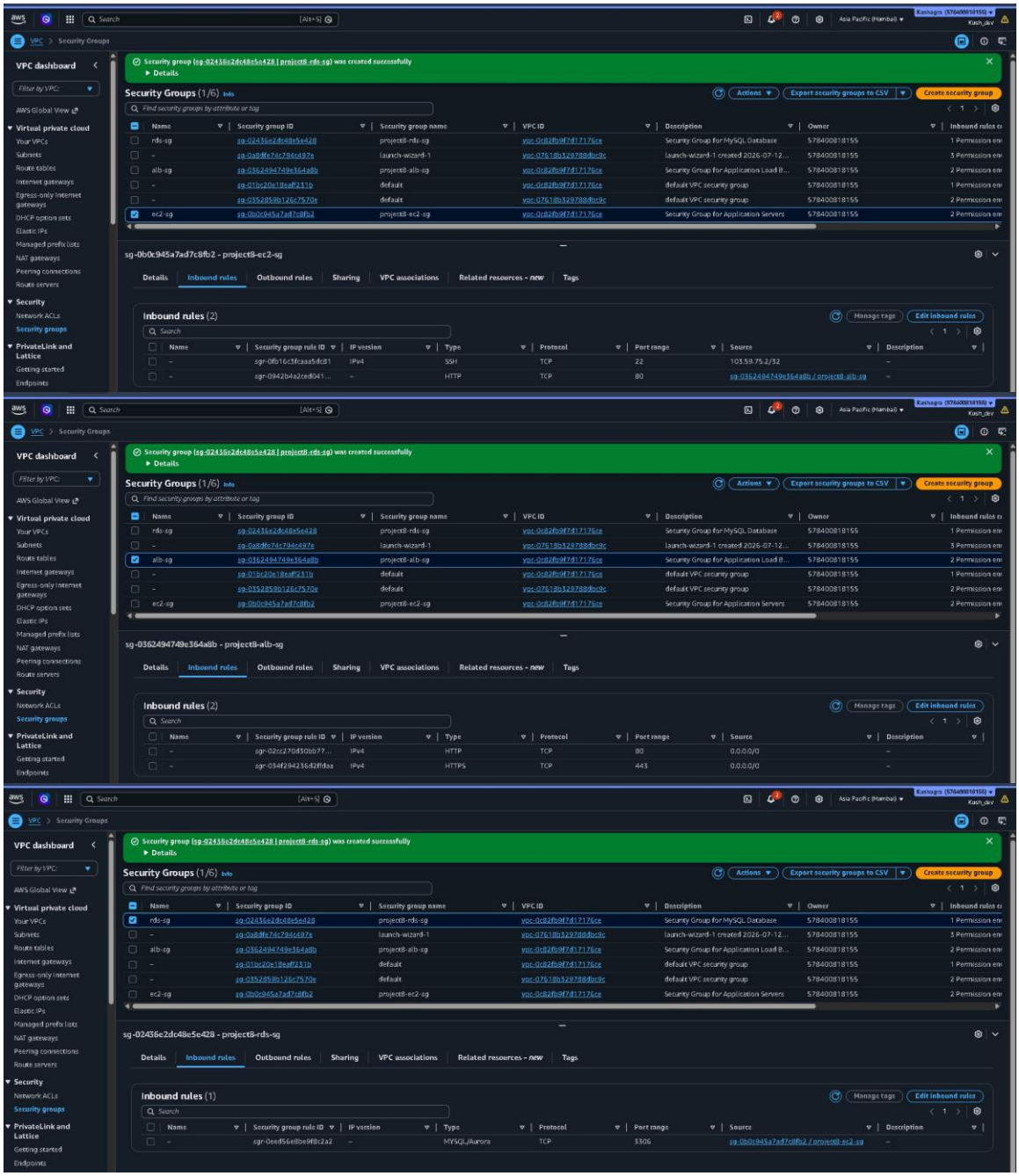

Figure 6: Security Group Configuration

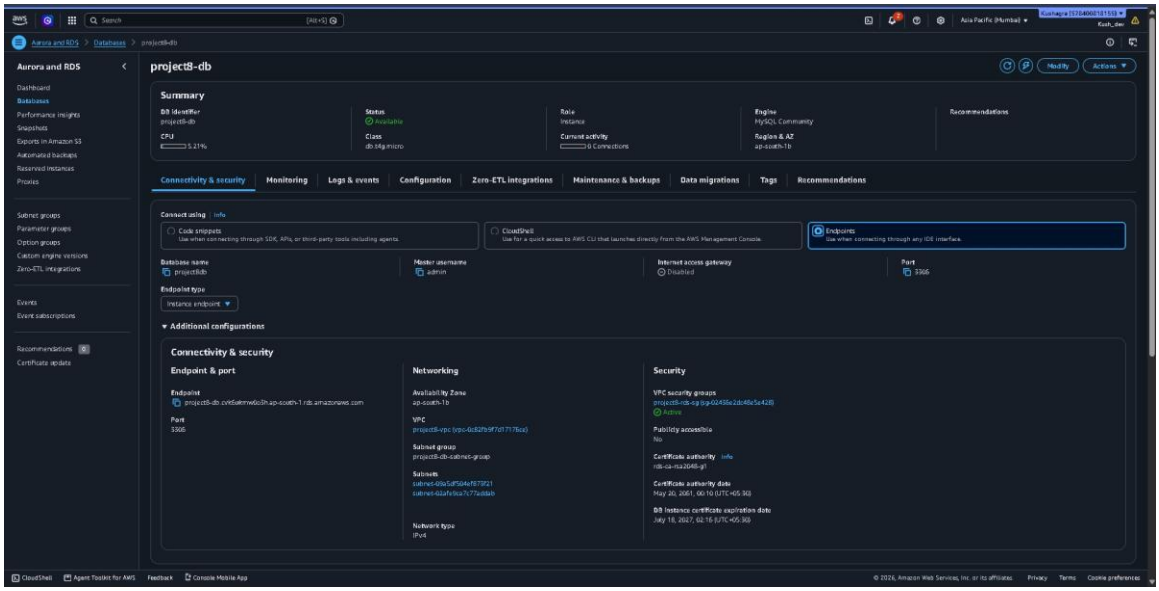

Figure 7: Amazon RDS Database Configuration

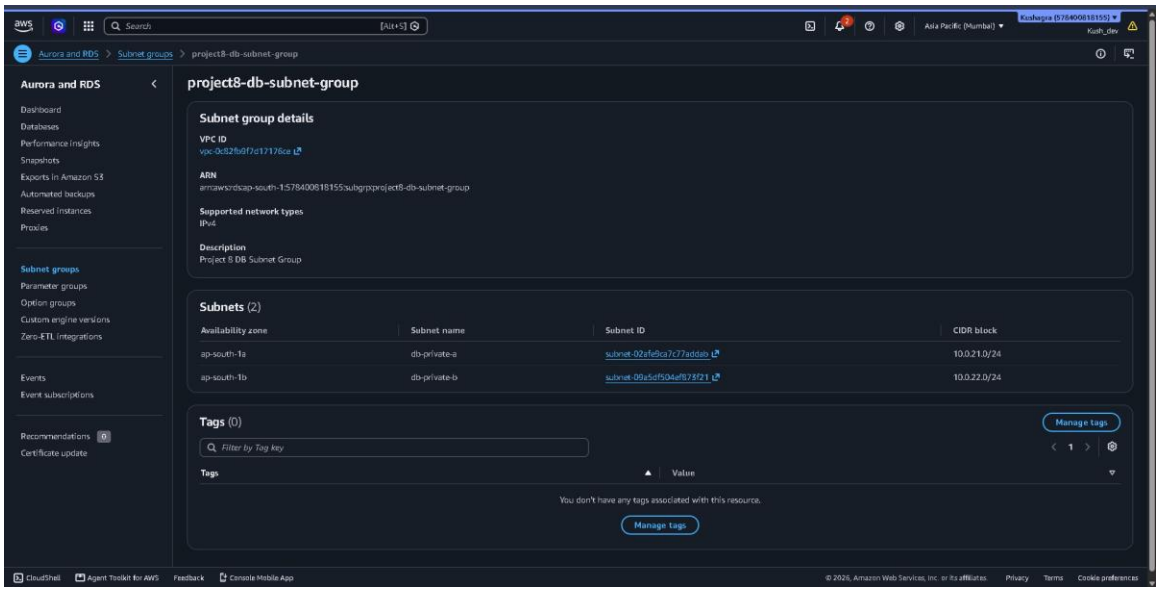

Figure 8: DB Subnet Group Configuration

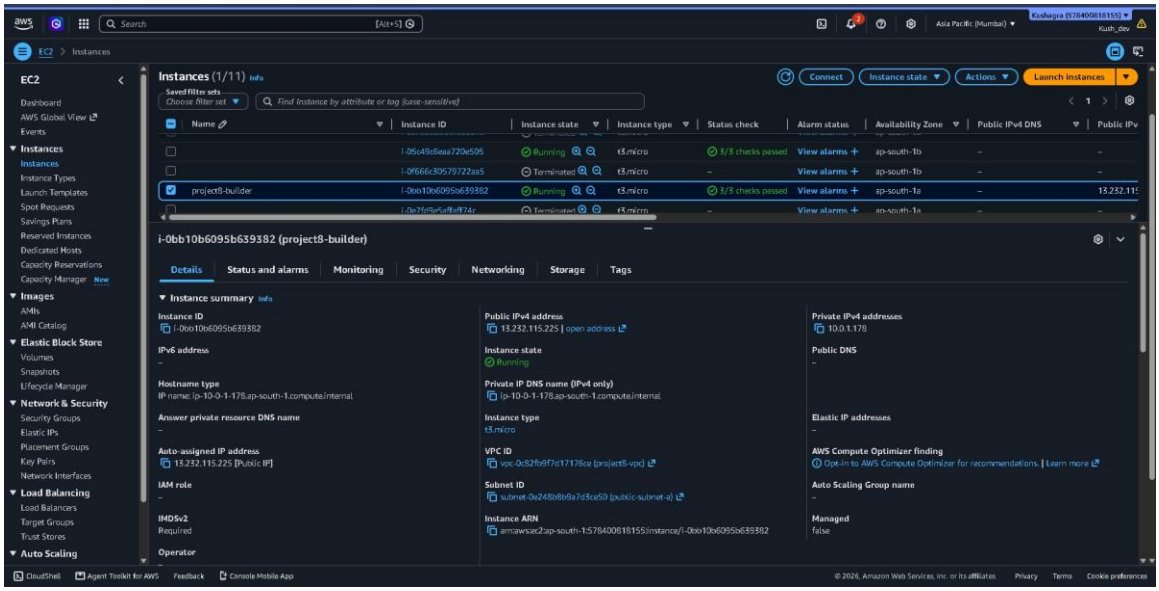

Figure 9: Builder EC2 Instance

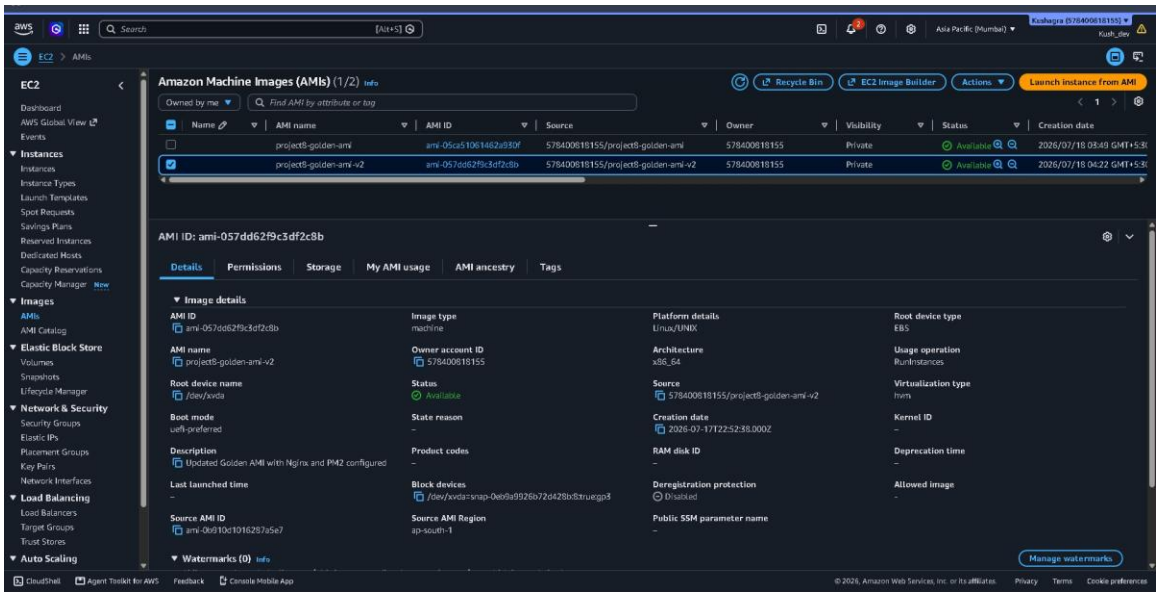

Figure 10: Golden AMI Creation

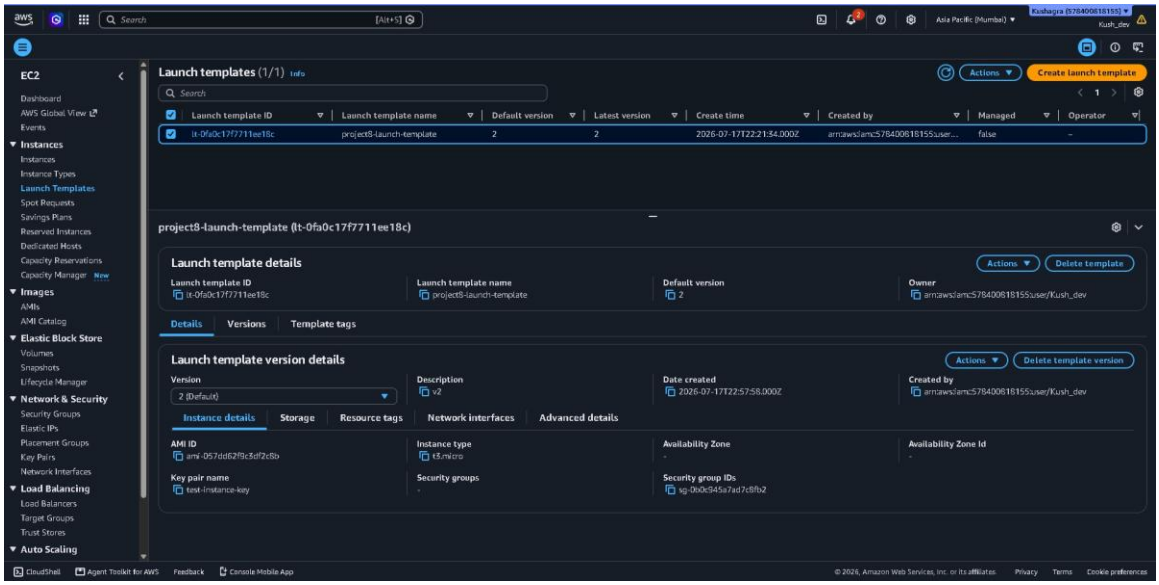

Figure 11: Launch Template Configuration

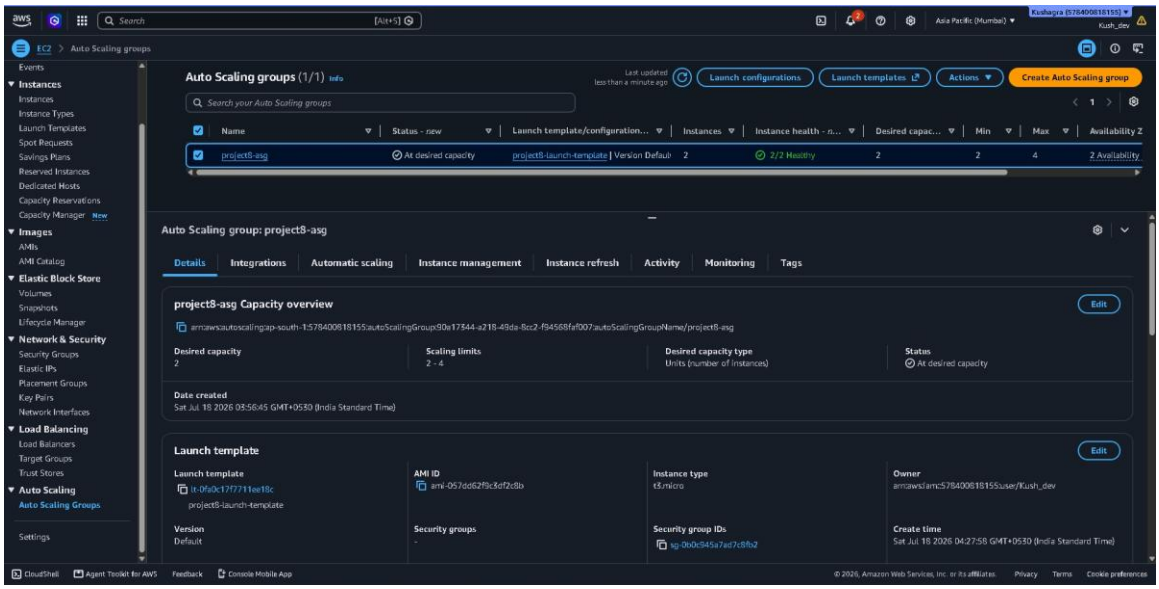

Figure 12: Auto Scaling Group Configuration

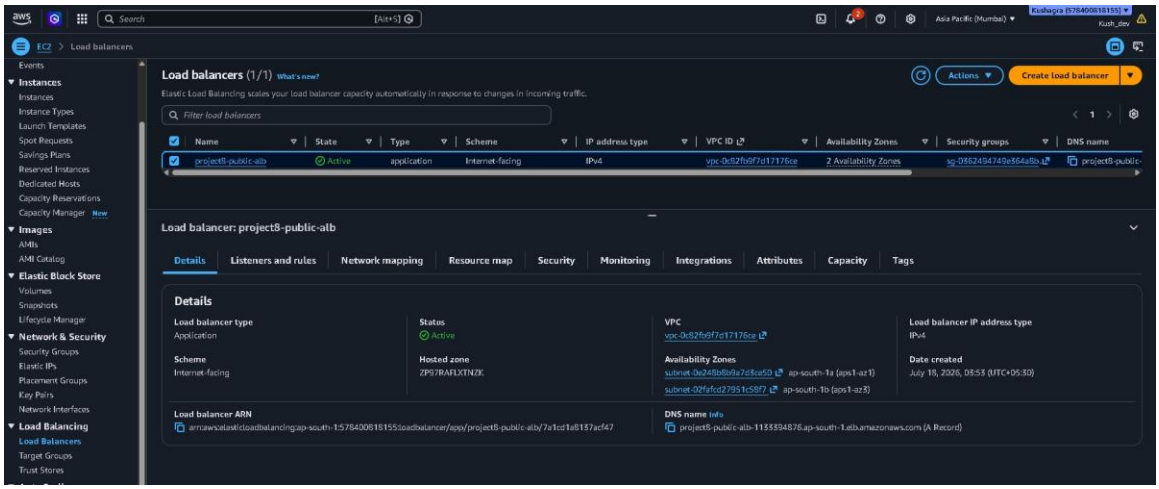

Figure 13: Application Load Balancer Configuration

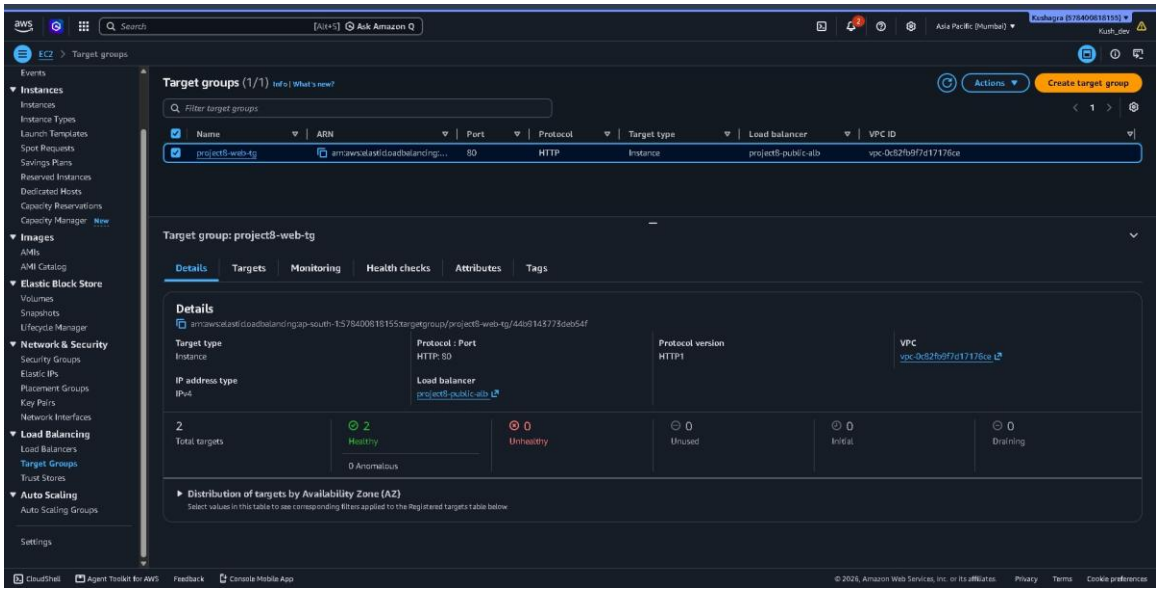

Figure 14: Target Group Health Status

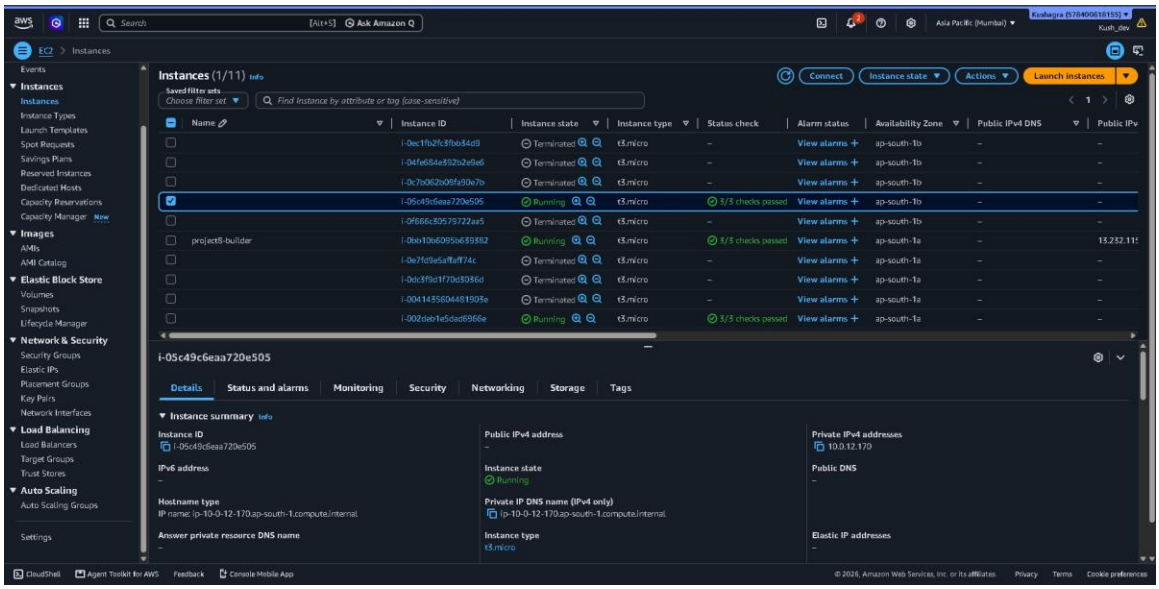

Figure 15: Auto Scalling EC2 Instances

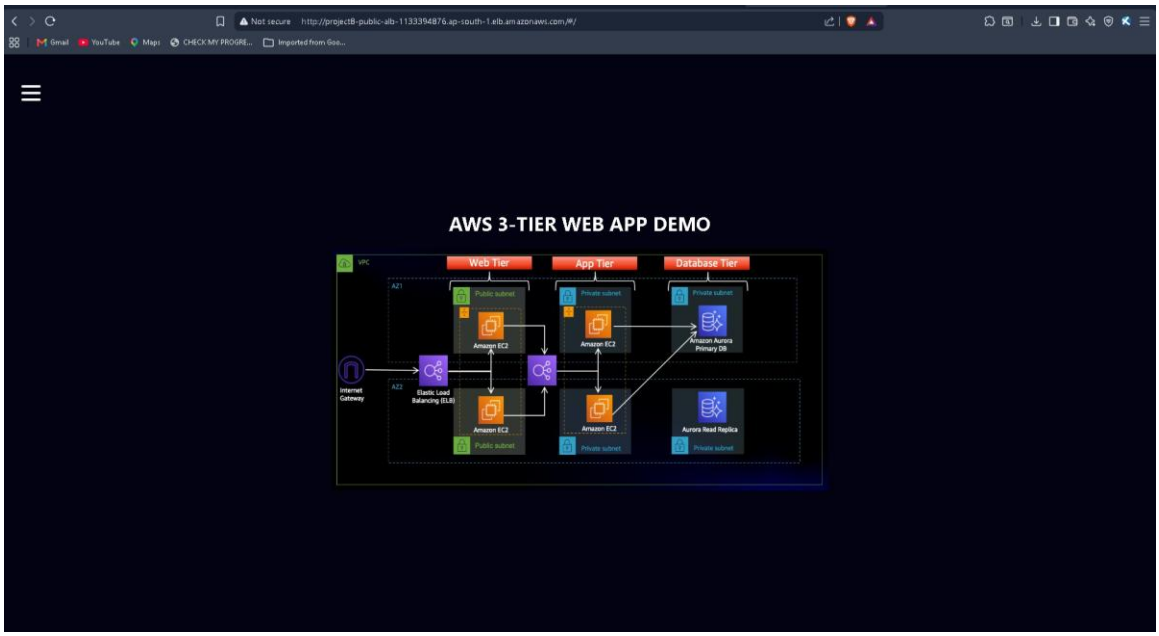

Figure 16: Website Home Page